

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital · Ingeniería de Software

Desarrollo de Aplicaciones Web

con ASP.NET Core 10

CRUD paso a paso con Visual Studio Code

Minimal API · Entity Framework Core · PostgreSQL · OpenAPI/Scalar · JWT

UTEQ · Unidad II · Dr. Gleiston Guerrero Ulloa, Ph.D. · PPA 2026–2027

UTEQ · Aplicaciones Web · Unidad II

Lo que veremos hoy

PARTE A · CONCEPTOS

- ¿Qué es ASP.NET Core?
- Historia: de ASP clásico a .NET 10
- C# para quien ya sabe Java
- El pipeline de middleware
- Inyección de dependencias
- Minimal API vs. controladores
- REST: rutas, verbos y códigos HTTP

PARTE B · CRUD PASO A PASO

- Crear el proyecto y los paquetes
- Modelo, DTO y DbContext
- Cadena de conexión y EF Core
- Migraciones a PostgreSQL
- Program.cs y endpoints CRUD
- Documentar con OpenAPI + Scalar
- Ejecutar, probar y asegurar

¿Qué es ASP.NET Core?

Un framework de Microsoft para construir aplicaciones y servicios web sobre la plataforma .NET, con el lenguaje C#.

- Multiplataforma: Windows, Linux y macOS
- Código abierto y de altísimo rendimiento
- Tipado fuerte con C#: errores en compilación
- Mismos patrones que Spring Boot o Laravel

EN UNA FRASE

«Una plataforma de pleno derecho para aplicaciones web modernas, multiplataforma y de alto rendimiento.»

— Andrew Lock

De ASP clásico a .NET 10



Punto de inflexión: 2016. El giro a código abierto y multiplataforma volvió a .NET competitivo frente a Java y Node.js.

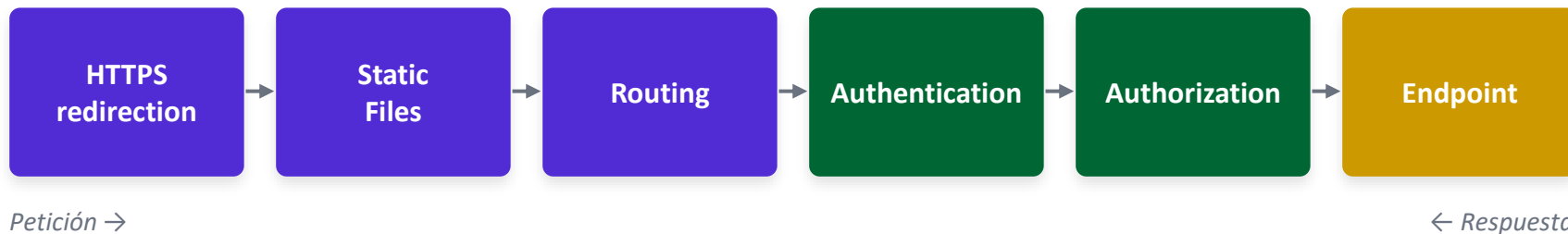
C# para quien ya sabe Java

Ambos son de tipado estático, orientados a objetos y de sintaxis con llaves. Estas son las diferencias que más verás:

Métodos y propiedades	PascalCase (mayúscula inicial)
Estado de las clases	Propiedades { get; set; } en vez de getters/setters
Inferencia de tipos	var nombre = "Ada";
Datos inmutables	record en una sola línea
Expresiones y lambdas	operador => para cuerpos y funciones
Asincronía	async / await liberan el hilo en E/S

El pipeline de middleware

Cada petición atraviesa una cadena de componentes (middleware). El orden en que los registras es el orden en que se ejecutan.



Regla de oro: UseAuthentication() va siempre antes que UseAuthorization(). Primero se establece quién es el usuario; solo después si tiene permiso.

Inyección de dependencias (DI)

Registras tus servicios una vez y el framework te los entrega donde los necesitas. Desacopla el código y facilita las pruebas.

Program.cs

```
// 1) Contrato e implementacion
public interface ISaludador { string Saludar(string n); }
public class Saludador : ISaludador
{
    public string Saludar(string n) => $"Hola, {n}!";
}

// 2) Registrar en el contenedor
builder.Services.AddScoped<ISaludador, Saludador>();

// 3) El framework lo inyecta en el endpoint
app.MapGet("/saludo/{n}", (string n, ISaludador svc)
    => svc.Saludar(n));
```

CICLOS DE VIDA

Singleton

una instancia para toda la app

Scoped

una por cada petición

Transient

una nueva cada vez

Minimal API vs. controladores

MINIMAL API

Rutas definidas directamente en Program.cs

- Conciso: todo a la vista
- Ideal para servicios pequeños
- Perfecto para aprender
- Lo usamos en el CRUD de hoy

CONTROLADORES

Clases con [ApiController] — como @RestController

- Organización por clases
- Ideal para APIs grandes
- Filtros y atributos aportan estructura
- Familiar si vienes de Spring

Verbos HTTP y códigos de respuesta

Operación	Verbo HTTP	Respuesta típica	Código
Leer lista / detalle	GET	Results.Ok(...)	200 OK
Crear	POST	Results.Created(...)	201 Created
Actualizar	PUT	Results.NoContent()	204 No Content
Eliminar	DELETE	Results.NoContent()	204 No Content
No encontrado	—	Results.NotFound()	404 Not Found
Validación fallida	—	Results.ValidationProblem(...)	400 Bad Request

Devolver el código correcto en cada caso es lo que distingue una API profesional de una improvisada.

PARTE B

CRUD paso a paso con Visual Studio Code

Una Web API REST de estudiantes, de principio a fin, con la CLI de dotnet y Entity Framework Core sobre PostgreSQL.

El stack y sus versiones (2026)



SDK .NET 10

LTS hasta 2028



C# 14

Lenguaje



ASP.NET Core 10

Framework web



EF Core 10

ORM



Npgsql 10

Proveedor PostgreSQL



PostgreSQL 18

Base de datos



VS Code + C# Dev Kit

IDE de referencia



OpenAPI + Scalar

Documentación

Crear el proyecto y los paquetes

EN VISUAL STUDIO CODE

- Abre una terminal en tu carpeta de prácticas
- Ejecuta los comandos de la derecha
- Abre la carpeta con code .
- Acepta confiar en la carpeta cuando lo pida

Terminal

```
dotnet new web -o ApiEstudiantes
cd ApiEstudiantes

dotnet add package \
  Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add package \
  Microsoft.EntityFrameworkCore.Design
dotnet add package Microsoft.AspNetCore.OpenApi
dotnet add package Scalar.AspNetCore

code .
```

Modelo, DTO y DbContext

Datos.cs

```
public class Estudiante
{
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
    public string Correo { get; set; } = "";
    public string Carrera { get; set; } = "";
    public DateTime Creado { get; set; }
}

// DTO: Lo que entra por La API
public record EstudianteEntradaDto(
    string Nombre, string Correo, string Carrera);

public class AppDbContext(DbContextOptions o)
    : DbContext(o)
{
    public DbSet<Estudiante> Estudiantes => Set<Estudiante>();
}
```

TRES PIEZAS

Modelo

la fila que se guarda en la tabla

DTO

lo que el cliente envía o recibe; no expone la tabla

DbContext

la sesión con la base de datos

Cadena de conexión y registro de EF Core

appsettings.json

```
{
  "ConnectionStrings": {
    "Postgres": "Host=127.0.0.1;Port=5432;
      Database=appweb2026ppa;
      Username=postgres;Password=P@$7Gr3$QL"
  }
}
```

Program.cs

```
builder.Services.AddDbContext<AppDbContext>(o
=>
    o.UseNpgsql(
        builder.Configuration
            .GetConnectionString("Postgres"),
        npg => npg.SetPostgresVersion(18,
0))));
```

Seguridad: nunca subas contraseñas al repositorio. En desarrollo usa User Secrets (dotnet user-secrets); en producción, variables de entorno. Aquí se deja visible solo con fines didácticos.

Migraciones: del modelo a las tablas

Una migración es un archivo versionado que describe los cambios de esquema. EF Core las genera comparando tu modelo y las aplica a la base.

Terminal

```
// Instalar la CLI de EF Core (una sola vez)
dotnet tool install --global dotnet-ef

// Generar el código de la migración (carpeta Migrations/)
dotnet ef migrations add Inicial

// Aplicar los cambios a la base de datos
dotnet ef database update
```

CODE-FIRST

La fuente de verdad es tu código C#. EF Core crea y versiona el esquema por ti; mantén una sola fuente de verdad para evitar conflictos.

Program.cs — configuración

Program.cs (parte 1 de 2)

```
using Microsoft.EntityFrameworkCore;
using Scalar.AspNetCore;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseNpgsql(builder.Configuration
        .GetConnectionString("Postgres"),
        npg => npg.SetPostgresVersion(18, 0)));

builder.Services.AddOpenApi(); // genera /openapi/v1.json
var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi(); // documento OpenAPI
    app.MapScalarApiReference(); // UI en /scalar/v1
}
app.UseHttpsRedirection();
```


Endpoints CRUD — leer y crear

Program.cs (parte 2 de 2)

```
var grupo = app.MapGroup("/v1/estudiantes");

// READ: Lista y detalle
grupo.MapGet("/", async (AppDbContext db) =>
    await db.Estudiantes.OrderBy(e => e.Id).ToListAsync());
grupo.MapGet("/{id:int}", async (int id, AppDbContext db) =>
    await db.Estudiantes.FindAsync(id) is Estudiante e
        ? Results.Ok(e) : Results.NotFound());

// CREATE
grupo.MapPost("/", async (EstudianteEntradaDto dto, AppDbContext db) =>
{
    var e = new Estudiante { Nombre = dto.Nombre, Correo = dto.Correo,
        Carrera = dto.Carrera, Creado = DateTime.UtcNow };
    db.Estudiantes.Add(e);
    await db.SaveChangesAsync();
    return Results.Created($"v1/estudiantes/{e.Id}", e);
});
```

Endpoints CRUD — actualizar y eliminar

Program.cs (continuación)

```
// UPDATE
grupo.MapPut("/{id:int}", async (int id,
    EstudianteEntradaDto dto, AppDbContext db) =>
{
    var e = await db.Estudiantes.FindAsync(id);
    if (e is null) return Results.NotFound();
    e.Nombre = dto.Nombre; e.Correo = dto.Correo;
    e.Carrera = dto.Carrera;
    await db.SaveChangesAsync();
    return Results.NoContent();
});

// DELETE
grupo.MapDelete("/{id:int}", async (int id, AppDbContext db) =>
{
    var e = await db.Estudiantes.FindAsync(id);
    if (e is null) return Results.NotFound();
    db.Estudiantes.Remove(e);
    await db.SaveChangesAsync();
    return Results.NoContent();
}); // y al final: app.Run();
```

Documentar con OpenAPI y Scalar

Cambio en .NET 9/10: la plantilla webapi ya NO trae Swagger. El documento lo genera el paquete integrado OpenApi; la UI se añade aparte.

URLs con la app en marcha

```
// Documento OpenAPI (JSON)
https://localhost:5xxx/openapi/v1.json

// Interfaz interactiva de Scalar
https://localhost:5xxx/scalar/v1
```

¿Para qué sirve?

Scalar genera una interfaz donde ves y pruebas cada endpoint desde el navegador, sin escribir cliente: ejecutas un GET, creas un estudiante con POST y compruebas los códigos 200, 201 y 404. Para abrirla automáticamente con F5, pon `launchBrowser true` y `launchUrl scalar/v1` en `launchSettings.json`.

Ejecutar y probar la API

Terminal

```
dotnet ef migrations add Inicial
dotnet ef database update
dotnet run
```

CÓMO PROBARLO

- Desde la interfaz de Scalar (recomendado)
- Con curl o la extensión REST Client de VS Code
- Verifica los datos en pgAdmin

Probar con curl

```
curl -X POST \
  https://localhost:5xxx/v1/estudiantes \
  -H "Content-Type: application/json" \
  -d '{ "nombre": "Ada",
        "correo": "ada@uteq.edu.ec",
        "carrera": "Software" }'
```

Listo: tienes una Web API REST completa, conectada a PostgreSQL y documentada. La base para tu backend con Angular.

Buenas prácticas y seguridad

JWT

Autenticación con tokens firmados; protege grupos con `RequireAuthorization`.

CORS

Autoriza solo los orígenes que consumen tu API (p. ej. Angular en `:4200`).

Validación

Valida toda entrada en el servidor; devuelve 400 con el detalle.

Capas

Extrae la lógica a servicios inyectables; deja los endpoints delgados.

Secretos

Contraseñas y claves fuera del código: User Secrets y variables de entorno.

Docker

Empaqueta la API con un Dockerfile multietapa para desplegar igual en cualquier lugar.

EN RESUMEN

De cero a una Web API REST completa y segura

Construiste, con Visual Studio Code y la CLI de dotnet, una API CRUD de estudiantes sobre PostgreSQL con EF Core, documentada con OpenAPI y Scalar.

Recursos: learn.microsoft.com/aspnet/core · npgsql.org/efcore · github.com/scalar/scalar